

Guía de la Actividad 4 — Técnicas, Metodologías y Conocimientos Requeridos

Materia: Ingeniería y Calidad de Software — 2026

Actividad: TP Integrador — Actividad 4: "Preparando para Producción (v2 Profesional)"

Propósito: Documentar qué se pide, qué técnicas aplica y qué conocimientos previos requiere

1. ¿Qué pide la Actividad 4?

Llevar la aplicación de un entorno de desarrollo Docker a una arquitectura de producción profesional con observabilidad 360°. Esto implica 5 fases secuenciales:

```
Fase 1: Analizar y proponer (individual)
      ↓
Fase 2: Especificar y diseñar (grupāl)
      ↓
Fase 3: Docker producción (grupāl – código)
      ↓
Fase 4: Observabilidad profesional (grupāl – código)
      ↓
Fase 5: Verificar y entregar (grupāl – informe + presentación)
```

Fase 1 — Analizar y proponer (individual)

- Analizar la infraestructura Docker actual del proyecto

2. Técnicas Aplicadas

2.1. Docker multi-stage

¿Qué es? Construir imágenes Docker en múltiples etapas donde cada etapa tiene un propósito específico y solo la última etapa va al runtime.

Problema que resuelve: Las imágenes de desarrollo incluyen herramientas de build (TypeScript compiler, npm, test runners) que NO son necesarias en producción. Esto produce imágenes de ~1GB cuando deberían ser ~250MB.

Cómo se aplica en la actividad:

```
Stage "deps":      Instalar solo dependencias de producción (npm ci --omit=dev)
Stage "build":     Compilar TypeScript (incluye dev deps para tsc, prisma generate)
Stage "runtime":   Solo node_modules prod + JS compilado + usuario no-root
```

Técnicas relacionadas:

3. Metodologías Aplicadas

3.1. Feature Branch Workflow

Cada cambio se desarrolla en una rama separada y se integra vía Pull Request.

```
feature/dockerfile-api    → Pull Request → main
feature/opentelemetry     → Pull Request → main
feature/dashboards        → Pull Request → main
```

Por qué se usa: Permite trabajo paralelo, revisión de código, y mantener `main` estable.

3.2. Conventional Commits

Mensajes de commit estructurados con tipo y scope:

```
feat(infra): agrega dockerfile multi-stage para API
fix(api): corrige healthcheck de nginx
docs(infra): mejora documentación de infraestructura
```

4. Conocimientos Previos Requeridos

4.1. Docker y Contenedores

Concepto	Nivel requerido	Para qué
<code>docker build</code> , <code>docker run</code> , <code>docker compose</code>	Alto	Trabajo diario
Multi-stage builds	Medio	Fase 3
Layer caching	Medio	Optimizar builds
Dockerfile: FROM, COPY, RUN, CMD, ENTRYPOINT	Alto	Escribir Dockerfiles
Docker Compose: services, volumes, networks	Alto	Orquestar servicios

5. Mapa de Aprendizaje

Cómo se relaciona la actividad con los contenidos de la materia:

Calidad de Software

- Análisis estático
 - Linting (ESLint) – actividad 1/2
 - TypeScript strict – actividad 1/2
- Testing
 - Unitario + Integración – actividad 1/2
 - E2E (Playwright) – actividad 2
 - Cobertura – actividad 2
- Infraestructura y Operaciones ← ACTIVIDAD 4
 - Docker multi-stage ← optimización de imágenes
 - Hardening de seguridad ← cap_drop, no-root, read-only
 - OpenTelemetry ← estándar de telemetría
 - Prometheus ← backend de métricas
 - Grafana ← visualización y dashboards
 - Alertas ← detección proactiva
 - Documentación técnica ← arquitectura y decisiones

6. Complejidad y Tiempo Estimado

Fase	Duración	Tipo	Dificultad
Fase 1: Analizar	2h	Individual	Media
Fase 2: Diseñar	3h	Grupal	Alta
Fase 3: Docker prod	4h	Grupal	Media
Fase 4: Observabilidad	6h	Grupal	Alta
Fase 5: Verificar	2h	Grupal	Media
Total	~17h		

Perfil de dificultad: La mayor dificultad está en Fase 4 (OpenTelemetry + dashboards) porque combina múltiples tecnologías nuevas. Fase 2 requiere pensamiento abstracto (diseñar sin implementar). Fase 3 es la más concreta y accesible.

7. Referencias

Docker

- <https://docs.docker.com/build/building/multi-stage/>
- <https://docs.docker.com/engine/security/>
- <https://docs.docker.com/compose/production/>

OpenTelemetry

- <https://opentelemetry.io/docs/languages/js/>
- <https://www.aspecto.io/blog/opentelemetry-metrics-guide/>

Observabilidad

- <https://grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/>